

Training a Small Language Model: nanoGPT + LoRA

Scaler School of Technology | Deep Learning I | Week 8 — Session 6/6 | Post-Lecture Review & Practice



Key Takeaways

1 Pretraining from scratch requires data proportional to model size

nanoGPT ($\sim 10\text{M}$ params) trained on $\sim 30\text{MB}$ of WikiText-103 produces coherent English prose. GPT-2 (117M params) was trained on $\sim 40\text{B}$ tokens of internet text, encoding vast world knowledge in its weights. Scale in both **model parameters** and **training tokens** drives quality — this is the core insight of modern LLM scaling laws.

2 LoRA updates $< 1\%$ of parameters to redirect pretrained knowledge

LoRA (Low-Rank Adaptation) freezes all original weights W and injects small trainable matrices: $W_{\text{new}} = W_{\text{frozen}} + BA$ where $A \in \mathbb{R}^{d \times r}$, $B \in \mathbb{R}^{r \times d}$, and $r \ll d$. For GPT-2 with $r = 8$: each 768×768 weight matrix adds only $2 \times 768 \times 8 = 12,288$ params. Total trainable: $\sim 700\text{K}$ out of 117M — just 0.6% .

3 Character-level vs subword tokenisation: a fundamental trade-off

nanoGPT uses character-level tokens — no library needed, fully transparent, ASCII vocabulary (~ 95 chars). GPT-2 uses Byte Pair Encoding (BPE) with $\sim 50\text{K}$ tokens — more efficient (fewer tokens per sentence), better handling of rare words, but requires a pretrained tokenizer. The vocabulary choice affects sequence length, model capacity requirements, and generation quality.

Test Your Understanding

Try answering these without looking at the slides. Answers are on page 3.

- Q1.** What is the fundamental difference between *pretraining from scratch* and *fine-tuning with LoRA*? When would you choose each approach?
- Q2.** LoRA decomposes a weight update $\Delta W \in \mathbb{R}^{d \times d}$ as BA where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times d}$. Why does this reduce the number of trainable parameters? Give the exact parameter counts for $d = 768$, $r = 8$.
- Q3.** nanoGPT uses pre-norm (LayerNorm *before* the sublayer) while the original Transformer used post-norm (LayerNorm *after*). What is the training stability benefit of pre-norm?
- Q4.** Your nanoGPT training loss decreases steadily but the generated text still has spelling errors after 1500 steps. What does this indicate about the model's learning stage, and what would you try next?
- Q5.** Explain why saving a LoRA adapter ($\sim 2\text{MB}$) is more practical for deployment than saving the full fine-tuned model ($\sim 500\text{MB}$). What must be present at inference time for the adapter to work?

Coding Exercises

Work through these in a Colab notebook with GPU enabled (Runtime > T4 GPU).

Exercise 1: Train nanoGPT and Track Perplexity

Train nanoGPT on WikiText-103 for 1500 steps using the notebook setup. At every 300 steps, record training loss, validation loss, and compute perplexity ($\text{PPL} = e^{\text{val.loss}}$). Plot both curves side by side. Generate 200 characters at steps 500 and 1500 using seed "The history of" with temperature 0.8. *Expected output:* dual loss + PPL plot + 2 text samples showing progression. *Target:* val PPL < 15 by step 1500.

Exercise 2: LoRA Rank Ablation Study

Fine-tune GPT-2 on the Python code dataset using LoRA with $r \in \{2, 8, 16\}$. For each rank: record trainable parameter count, training time (1 epoch), and generate a 100-token Python snippet starting with "def train_model("). Create a comparison table: Rank | Trainable Params | Train Time | Qualitative Quality. *Expected output:* comparison table + 3 code snippets. *Observe:* does quality saturate before $r = 16$?

Exercise 3: nanoGPT vs Char-LSTM Head-to-Head

Session 3 trained a CharLSTM on Tiny Shakespeare; this session trains nanoGPT on WikiText-103. Retrain the CharLSTM on WikiText-103 (same 30MB subset) so the comparison is fair. Compare: (a) validation loss after equivalent training time, (b) generated text quality from the same seed and temperature, (c) time per training step on GPU. *Expected output:* side-by-side text samples + loss/speed comparison table showing Transformer > LSTM.

Challenge Problem

Build a Domain-Specific Assistant with LoRA:

1. Choose a domain text corpus: movie scripts, news articles, scientific abstracts, or song lyrics (find a ~1MB dataset online)
2. Fine-tune GPT-2 on your corpus using LoRA ($r = 8$, same config as the notebook)
3. Evaluate: generate 5 samples of 150 tokens each from diverse prompts
4. Compare generation quality against the base GPT-2 (without LoRA) on the same prompts
5. Save the adapter and measure its file size

Bonus: Try loading two different LoRA adapters (Python code adapter + your domain adapter) and observe whether they can be blended by averaging the A, B matrices.

Further Reading

- **LoRA:** Hu et al. (2022), “*LoRA: Low-Rank Adaptation of Large Language Models*” — the paper that made efficient fine-tuning practical; focus on Sections 2–4.
- **Scaling Laws:** Kaplan et al. (2020), “*Scaling Laws for Neural Language Models*” — shows how loss scales predictably with model size, data, and compute.
- **Chinchilla:** Hoffmann et al. (2022), “*Training Compute-Optimal Large Language Models*” — revised scaling laws showing data is as important as model size.
- **nanoGPT:** Karpathy (2022), “*nanoGPT*” (GitHub) + accompanying “Let’s build GPT” YouTube lecture — the reference implementation for this session.

Online Resources & Tools

- **HuggingFace PEFT library** — LoRA in 5 lines
- **HuggingFace transformers** — GPT-2 loading
- **Karpathy’s nanoGPT** — reference impl.
- **Weights & Biases** — LM training tracking
- **tiktoken** — GPT tokenizer library
- **LM Evaluation Harness** — standard benchmarks

Fill the Gaps — Prerequisite Refreshers

Felt lost on a specific concept? These resources will help you catch up.

- **Unclear on decoder-only vs encoder-decoder Transformers?** Revisit Session 5 handout page 4 — decoder-only (GPT) generates autoregressively using only masked self-attention; no cross-attention to a separate encoder.
- **Confused about BPE tokenisation?** HuggingFace tokenizers docs — BPE iteratively merges the most frequent byte pairs, building a vocabulary of subword units that balances coverage and efficiency.
- **What is rank in matrix decomposition?** Khan Academy linear algebra — a rank- r matrix has at most r linearly independent rows/columns. LoRA uses $r \ll d$ to constrain the expressivity of the weight update.
- **Forgot what perplexity means?** Session 3 handout page 5 — $\text{PPL} = e^H$ where H is cross-entropy loss. Lower PPL = model is less surprised by the text.

Answers — Test Your Understanding

- A1.** *Pretraining:* random weights, train on large corpus from scratch — massive data/compute required. *LoRA:* start from pretrained weights, inject small adapters to redirect to a new domain with minimal compute. Choose scratch when no suitable pretrained model exists; choose LoRA otherwise.
- A2.** Full ΔW : $768 \times 768 = 589,824$ params. *BA:* A is $8 \times 768 = 6,144$ and B is $768 \times 8 = 6,144$ — total 12,288 per matrix. Reduction: $\approx 48\times$ fewer trainable parameters.
- A3.** Pre-norm applies LayerNorm to the residual stream *before* the sublayer, stabilising gradient magnitudes through depth. Post-norm normalises after adding the residual, scaling gradient paths unevenly and causing instability at large depths. Pre-norm trains reliably without warmup tuning.
- A4.** Decreasing loss confirms learning, but spelling errors at 1500 steps are expected — the model is still in early word-level learning on a large diverse corpus. Try: increase MAX.STEPS to 3000, verify the ASCII filter is applied (unicode chars cause garbage), and check the loss is below 2.5 by step 500.
- A5.** The adapter ($\sim 2\text{MB}$) stores only the A, B matrices. The frozen base model ($\sim 500\text{MB}$) must also be present at inference: $W_{\text{eff}} = W_{\text{frozen}} + BA$. One base model can host many domain adapters, making distribution and storage highly efficient.

Deep Dive: LoRA — The Mathematics

For a pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times d}$, LoRA constrains the update to a low-rank subspace:

$$h = W_0 x + \Delta W x = W_0 x + B A x, \quad \Delta W = \frac{\alpha}{r} \cdot B A$$

$A \in \mathbb{R}^{r \times d}$ is initialised $\mathcal{N}(0, \sigma^2)$; $B \in \mathbb{R}^{d \times r}$ is initialised to $\mathbf{0}$. At init, $\Delta W = \mathbf{0}$ so training starts from exact pretrained predictions. The α/r scaling keeps update magnitude constant as r varies. The original paper applies LoRA to W^Q and W^V ; adding keys, output projections, and FFN layers increases quality at the cost of more parameters.

Deep Dive: Scaling Laws — What Governs LLM Quality?

Kaplan et al. (2020) showed loss follows smooth power laws: $L(N) \approx (N_c/N)^{\alpha_N}$, similarly for training tokens D and compute C .

Variable	nanoGPT (session)	GPT-3
Parameters N	$\sim 3\text{M}$	175B ($\times 58\text{K}$)
Training tokens D	$\sim 1\text{M}$	300B ($\times 300\text{K}$)
Compute C	$\sim 10^{10}$ FLOPs	$\sim 3 \times 10^{23}$ FLOPs

Chinchilla (2022): For a given compute budget, train a *smaller* model on *more* data — data and parameters should scale equally.

Deep Dive: nanoGPT Architecture in Detail

Each decoder block (pre-norm order):

$$x \leftarrow x + \text{CausalSelfAttn}(\text{LN}(x)), \quad x \leftarrow x + \text{MLP}(\text{LN}(x))$$

Session config: `n_layer=6`, `n_head=8`, `n_embd=512`, `block_size=256`, `vocab_size=~95`. Parameter breakdown: token embedding ($95 \times 512 = 49\text{K}$) + positional ($256 \times 512 = 131\text{K}$) + 6 blocks ($\approx 10\text{M}$) $\approx$ **10M total**. Trains in ≈ 8 min on Colab T4.

Guided Experiments

1. **nanoGPT capacity sweep:** Train four configs: $(n_layer=2, n_embd=128)$, $(4, 256)$, $(6, 384)$, $(8, 512)$. Plot val loss vs parameter count. *Hypothesis:* loss decreases until data becomes the bottleneck.
2. **LoRA target module expansion:** Start with `c_attn` only; add `c_proj`, then `mlp.c_fc`. Record trainable params and generation quality per config.
3. **Context length effect:** Train nanoGPT with `block_size` $\in \{64, 128, 256, 512\}$. Does quality improve monotonically? At what length does Colab OOM?
4. **Temperature vs top-p:** Generate 10 samples with temperature ($T = 0.8$) and nucleus sampling ($p = 0.9$). Compare diversity and coherence qualitatively.

Try a Different Corpus

Swap out Tiny Shakespeare in Part 1 and observe how corpus style shapes generation. All are free, ~1–3MB, single-file downloads:

Corpus	Style	Why Interesting
Sherlock Holmes (Doyle)	Dialogue-heavy prose	Recognisable voice; outputs are immediately fun to read
ArXiv ML abstracts	Technical/formal	Generates fake-but-plausible research papers
Friends TV scripts	Casual dialogue	Learns conversational patterns quickly; funny outputs
US Presidential speeches	Formal rhetoric	Structured and repetitive — good for small models
Wikipedia Simple English	Encyclopedic	Controlled vocab; easy to spot factual errors in output

The Modern LLM Landscape

Model	Params	Tokens	Key Innovation	Year
nanoGPT (ours)	10M	30M chars	Minimal GPT implementation	2022
GPT-2 (LoRA'd)	117M	40B	Scaled unsupervised pretraining	2019
GPT-3	175B	300B	Few-shot in-context learning	2020
InstructGPT	175B	300B+	RLHF for instruction following	2022
LLaMA-2	7–70B	2T	Open weights, efficient training	2023
Mistral-7B	7B	8T	Grouped-query attention, sliding window	2023

PEFT Methods Beyond LoRA

- **Prefix Tuning:** Prepend learnable soft-prompt tokens to the input ($\sim 0.1\%$ params). Less stable than LoRA on small datasets.
- **Adapter Layers:** Bottleneck MLP ($d \rightarrow r \rightarrow d$) inserted between sublayers. More params than LoRA, adds slight inference latency.
- **QLoRA:** Quantize base model to 4-bit, then apply LoRA. Fine-tunes 65B models on a single 48GB GPU — standard for production LLM fine-tuning.
- **DoRA:** Decomposes ΔW into magnitude and direction components; often outperforms LoRA at the same rank.

nanoGPT Configuration Reference

Param	Session	Effect	How to Scale
n_layer	6	Transformer blocks	Increase first for quality
n_head	8	Attention heads	Must divide n_embd
n_embd	512	Model dimension	Powers of 2 preferred
block_size	256	Context window	Memory scales $O(n^2)$
dropout	0.2	Regularisation	Reduce for larger datasets
learning_rate	1e-3	Peak LR	Use cosine decay
max_iters	1500	Training steps	Increase until val plateau
batch_size	64	Sequences/step	Larger = more stable

LoRA Hyperparameter Guide

Parameter	Range	Effect	Recommendation
Rank r	4–64	Adapter capacity	Start with 8; sweep if budget allows
α	Usually $= r$	Scaling factor	Set equal to r as default
Dropout	0.05–0.1	Regularisation	0.05 for most tasks
Target modules	q, v to all	Layers with LoRA	Add more if quality plateaus
Learning rate	1e-4–3e-4	Adapter LR	Higher than full fine-tuning
Epochs	3–5	Duration	Monitor val loss for stopping

Debugging Checklist

Symptom	Likely Cause	Fix
Loss stuck at $\ln(65) \approx 4.17$	Model not updating	Verify <code>model.train()</code> ; check optimizer
Generates only spaces/newlines	Bad temperature or seed	Set $T = 0.8$; check char encoding
LoRA trainable params = 0	PEFT config missing	Call <code>get_peft_model()</code> first
GPT-2 outputs generic text	Too few epochs or low r	Train 3+ epochs; try $r = 16$
OOM on Colab	Batch size too large	Reduce to 4–8; use gradient checkpointing
Adapter loads but no change	Wrong base model	Match GPT-2 variant at train & inference

Notebooks: [Session 6A nanoGPT From Scratch.ipynb](#) | [Session 6B GPT2 LoRA Finetuning.ipynb](#)

Instructor:

Priyansh Saxena